

Guia Completo: Base Sólida para React App com Vite

1. Configuração do Repositório Git

1.1 Estrutura de Repositórios

```
git init meu-projeto-react  
cd meu-projeto-react
```

Configurar .gitignore com:

```
node_modules/, /dist, /build, .env*, .DS_Store, logs
```

1.2 Estrutura de Branches

- **main** - produção (protegida)
- **develop** - desenvolvimento principal
- **feature/*** - novas funcionalidades
- **bugfix/*** - correções de bugs
- **hotfix/*** - correções urgentes

1.3 Convenção de Commits

feat, fix, docs, style, refactor, test, chore

2. Quadro Trello - Estrutura Recomendada

2.1 Listas Principais

1. Backlog - ideias e funcionalidades futuras
2. To Do - tarefas priorizadas
3. In Progress - em desenvolvimento
4. Code Review - aguardando revisão
5. Testing - em testes
6. Done - concluído

2.2 Labels Sugeridas

Prioridade Alta/Média/Baixa, Bug, Feature, Documentação, Configuração, UI/UX

3. Criação do Projeto Vite

```
npm create vite@latest meu-projeto-react -- --template react
cd meu-projeto-react
npm install
```

4. Arquitetura de Pastas

```
meu-projeto-react/  
■ ■ ■ public/  
■ ■ ■ src/  
■ ■ ■ ■ assets/ (imagens, fontes, ícones)  
■ ■ ■ ■ components/ (componentes reutilizáveis)  
■ ■ ■ ■ common/ (Button, Input, Card)  
■ ■ ■ ■ layout/ (Header, Footer, Sidebar)  
■ ■ ■ ■ features/ (específicos de funcionalidade)  
■ ■ ■ ■ pages/ (páginas da aplicação)  
■ ■ ■ ■ hooks/ (custom hooks)  
■ ■ ■ ■ services/ (chamadas API)  
■ ■ ■ ■ store/ (estado global)  
■ ■ ■ ■ context/  
■ ■ ■ ■ reducers/  
■ ■ ■ ■ routes/ (configuração de rotas)  
■ ■ ■ ■ utils/ (funções utilitárias)  
■ ■ ■ ■ styles/ (estilos globais)  
■ ■ ■ ■ config/ (configurações)  
■ ■ ■ ■ App.jsx  
■ ■ ■ ■ main.jsx  
■ ■ ■ .env.example  
■ ■ ■ vite.config.js  
■ ■ ■ package.json
```

5. Configurações Essenciais

5.1 Dependências Principais

Roteamento: react-router-dom

Estado: zustand ou @reduxjs/toolkit

HTTP: axios

Formulários: react-hook-form, zod

Styling: tailwindcss ou styled-components

Utilitários: clsx, date-fns

Instalação:

```
npm install react-router-dom axios zustand react-hook-form zod
```

5.2 ESLint e Prettier

```
npm install -D eslint eslint-plugin-react eslint-plugin-react-hooks prettier
```

5.3 Path Aliases (vite.config.js)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
      '@components': path.resolve(__dirname, './src/components'),
      '@pages': path.resolve(__dirname, './src/pages'),
      '@hooks': path.resolve(__dirname, './src/hooks'),
      '@services': path.resolve(__dirname, './src/services'),
      '@utils': path.resolve(__dirname, './src/utils'),
    },
  },
  server: { port: 3000, open: true },
})
```

5.4 Variáveis de Ambiente

Criar arquivos .env.example e .env:

```
VITE_API_URL=http://localhost:8000/api
VITE_APP_NAME=Meu Projeto
VITE_ENV=development
```

Criar src/config/environment.js para centralizar acesso:

```
export const config = { apiUrl: import.meta.env.VITE_API_URL, ... }
```

5.5 Scripts do package.json

```
dev, build, preview, lint, lint:fix, format, test
```

6. Estrutura de Componentes

Padrão recomendado para cada componente:

- Arquivo principal (ComponentName.jsx)
- Estilos (ComponentName.module.css)
- Export (index.js)

```
// src/components/common/Button/Button.jsx
import styles from './Button.module.css'

export const Button = ({ children, variant = 'primary', onClick, disabled }) => {
  return (
    <button className={`${styles.button} ${styles[variant]}`} onClick={onClick}>
      {children}
    </button>
  )
}

// src/components/common/Button/index.js
export { Button } from './Button'
```

7. Configuração de Rotas

```
// src/routes/AppRoutes.jsx
import { BrowserRouter, Routes, Route } from 'react-router-dom'

export const AppRoutes = () => {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  )
}
```

8. Checklist de Implementação

- Repositório Git inicializado
- Quadro Trello configurado com listas e labels
- Projeto Vite criado
- Estrutura de pastas implementada
- ESLint e Prettier configurados
- Path aliases configurados no vite.config.js
- Variáveis de ambiente configuradas (.env)
- Dependências principais instaladas
- README.md documentado
- Scripts do package.json configurados
- Primeiro commit realizado

9. Boas Práticas

9.1 Organização de Código

- Um componente por arquivo
- Nomes descritivos e em PascalCase para componentes
- Separar lógica de apresentação
- Manter componentes pequenos e focados

9.2 Performance

- Usar React.memo para componentes que renderizam frequentemente
- Lazy loading de rotas e componentes pesados
- Otimizar imagens e assets

9.3 Segurança

- Nunca commitar arquivos .env
- Validar inputs do usuário
- Sanitizar dados antes de renderizar

10. Próximos Passos

1. Configurar testes (Vitest ou Jest)
2. Implementar CI/CD
3. Configurar análise de código (SonarQube)
4. Documentar componentes (Storybook)

5. Configurar monitoramento de erros (Sentry)